Loading Library Files in C++

This article demonstrates how to load shared or dynamic library files in programs written in C++, which is not as straightforward as in C.

Device drivers and library files have always been associated with the C programming language, and dominated by C programmers, because of the straightforward symbols of C's libraries, which are directly related to the function name. Loading a library in C is simpler than in C++, mainly due to the issue of name mangling, which we will examine later. Another problem of using *dlopen* in C++ is that the *dlopen* API supports loading of functions, but in C++, to use the methods of a class, normally, you need to instantiate it.

Name mangling

In any executable or shared library, all non-static functions are represented by a symbol, which is usually the same as the function name, and represents the start address of the function in memory. In C, the symbol is the same as the function name—e.g., the symbol for the *init* function will be *init*, since no two functions can have the same name.

However, in C++, because of overloading and polymorphism in classes, it is possible to have the same name for two functions in a program. Hence, it's not possible to use the function name as the symbol. To solve this problem of having two functions with the same name, C++ compilers use name-mangling techniques, in which they change the symbol names (you can read more about this at Wikipedia). Name mangling makes it difficult for programmers to access a specific symbol in the compiled shared library file, even if they know the original function name.

The solution to this issue is to use the special keyword *extern "C"* before the function implementation (i.e., *extern "C" void function_name()*). This tells the C++ compiler to compile the function in C style—to keep the symbol name the same as the function name. We can use this keyword to define a function that returns an instance of a class, which also solves the must-instantiate-class problem mentioned earlier.

Now, let's look at an example of building a library, and then loading it. First, let's make the interface for the shared library, which we can use to reference the shared library and our main programs:

```
#ifndef TESTVIR H
#define TESTVIR_H

class TestVir
{
```